

HeapSort(arr)

- ConvertToMaxHeap(arr, n)
- Set lastChildPos to n - 1
- while (lastChildPos > 0)
  - Swap root(0) and lastChildPos values
  - if (lastChildPos > 1)
    - MakeMaxHeap(arr, 0, lastChildPos)
  - Decrement lastChildPos by 1
- Stop

Time Complexity  $\frac{n \log_2 n + (n-1) \times \log_2 n}{2}$  Space  $O(1)$

$(n-1) (n-2) \dots 1$

terms?  $(n-1)$  times

$$= n \log_2 n + n \log_2 n$$

$$- \log_2 n$$

$$= 2n \log_2 n - \log_2 n$$

||

$$O(n \log_2 n)$$

space

↓

iter → 1

or

$$\log_2 n$$

↑ recursive

ConvertToMaxHeap(arr, n)

- Set lastParent to n / 2 - 1
- while (lastParent >= 0)
  - MakeMaxHeap(arr, lastParent, n)
  - Decrement lastParent by 1
- Stop

$(\frac{n}{2} - 1) (\frac{n}{2} - 2) \dots 0$

terms?  $\frac{n}{2}$  times

$$\log_2 n$$

MakeMaxHeap(arr, parent, n) 

// Find which child has largest value

- Set maxChildPos to  $2 * \text{parent} + 1$
- Set rightChildPos to  $2 * \text{parent} + 2$

// If right child exist, is it the one with value larger of two childs?

- if ( $\text{rightChildPos} < n$ )
  - if ( $\text{arr}[\text{rightChildPos}] > \text{arr}[\text{maxChildPos}]$ )
    - $\text{maxChildPos} = \text{rightChildPos}$

// Check if parent has value larger than the largest child

- if ( $\text{arr}[\text{parent}] > \text{arr}[\text{maxChildPos}]$ )
  - Stop

// As child has larger value, swap it with parent

- Swap values at parent and maxChildPos

// Child value has changed, if its a parent node then it should still be a max heap

- if ( $\text{maxChildPos}$  is a leaf node)
  - Stop
- MakeMaxHeap(arr, maxChildPos, n)
- Stop

Time  $\Rightarrow \log_2 n$   
Space  $\Rightarrow 1$

$\uparrow$   
if tail  
recursion is  
removed

$\log_2 n$   $\leftarrow$  with  
tail  
recursion

Merge Sort (arr, start, end)

→ if ( (end - start + 1)  $\leq$  ~~1~~<sup>20</sup> )  
→ ~~Stop~~ insertion Sort (arr, start, end)

→ mid = . . .

→ Create Thread (Merge Sort (arr, start, mid))

→ Create Thread (Merge Sort (arr, mid + 1, end))

wait for threads to get over

→ Merge (arr, start, mid, mid + 1, end)

Stable Sorting

Bubble  
Insertion  
Merge

|              |              |              |              |              |
|--------------|--------------|--------------|--------------|--------------|
| <div>N</div> | <div>A</div> | <div>B</div> | <div>C</div> | <div>D</div> |
| 4            | 1            | 1            | 2            | 3            |

Sort a string of characters (lower case)

str = "a b x y d e c a x c d y"

Time complexity =  $O(n \log_2 n)$

Counting Sort

int[] freq = new int[26];

n times  
for (char ch: str)  
++ freq[ch - 'a'];  
int k = 0;  
for (int i = 0; i < freq.length; ++i) {  
26 times if (freq[i] != 0) {

Time complexity

$$\Rightarrow n + n$$

$$= 2n$$

$$\Rightarrow O(n)$$

Space complexity

$$\Rightarrow O(1)$$

total times  
 $= n$

```
for (int j=0;
    j < freq[i];
    ++j) {
    str[i] = i + 'a';
    ++i;
}
```

}

# Hash Table

Used for efficient searching.

|              | Search                           | Insert        |
|--------------|----------------------------------|---------------|
| Sorted Array | $O(\log_2 n)$<br>(Binary Search) | $O(n)$        |
| Linked list  | $O(n)$<br>(Linear Search)        | $O(n)$        |
| BST          | $O(\log_2 n)$<br>(Binary Search) | $O(\log_2 n)$ |

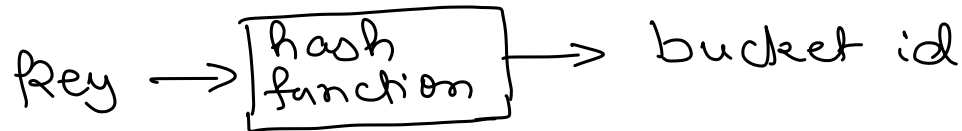
Hash Table search and insert time complexity is  $O(1)$ \*

\* Under ideal scenario.

Hash table is a collection of buckets.

Bucket is where key/value is stored.

Hash Function is a function that maps a key to a bucket. This process is called hashing.



# Types of Hash Table



Static

e.g. Symbol Table

Table of Key words

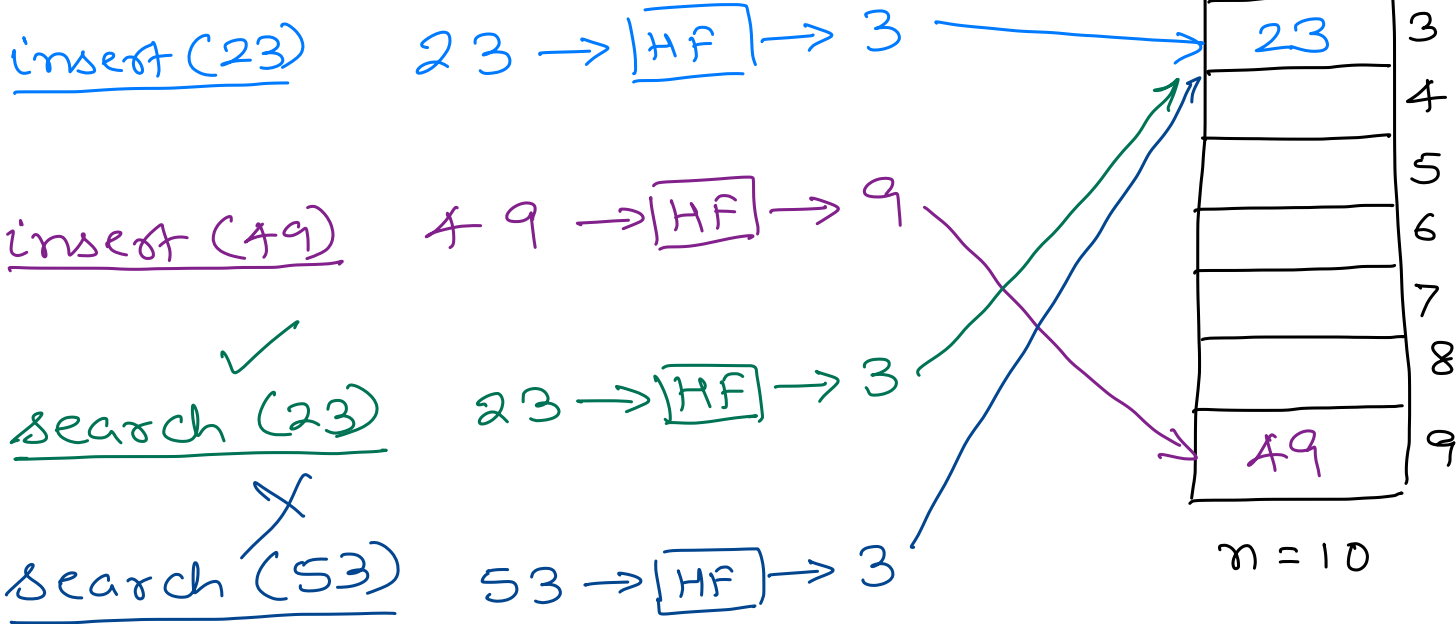


Dynamic

e.g. JAVA util  
library class.

Hash table using one dimensional array  
(bucket size = 1) of size 10.

Using MOD N as hash function.

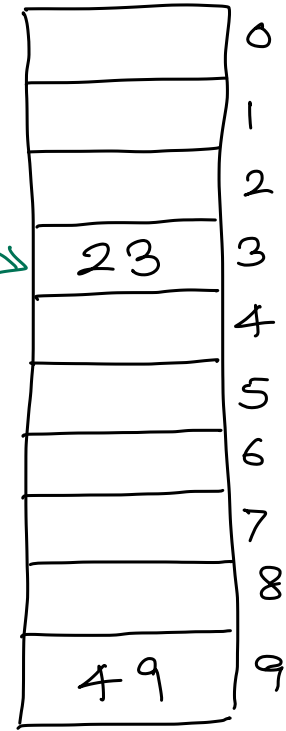


Insert (33)

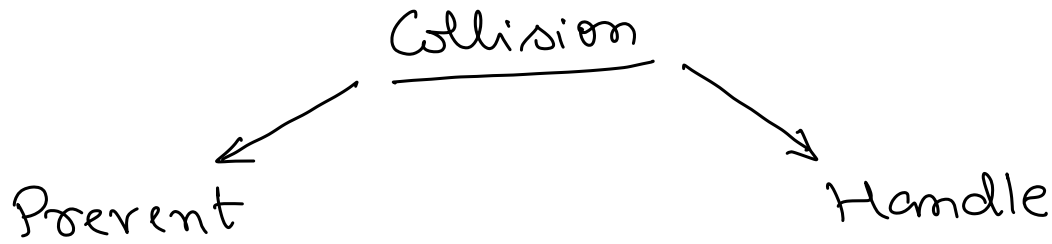
33  $\rightarrow$  HF  $\rightarrow$  3

Collision

When multiple keys are mapped to the same bucket.



$n = 10$



How to avoid collision?

→ use a better hash function.

① MOD N

② Folding → Break key into multiple parts and fold them.

43791  
└─┬─┬─┬─┐  
  43791

    4  
  37  
+ 91  
───  
132

③ Mid Square  $\rightarrow$  Find square of key and pick few digits from middle.

12  $\rightarrow$  square  $\rightarrow$  144

How to convert string to number?

- ① add ASCII value of each string character.
- ② use place value along with above approach.

"abc"  $\xrightarrow{\textcircled{1}}$   $97 + 98 + 99 = 294$

$\xrightarrow{\textcircled{2}}$   $97 \times 10^2 + 98 \times 10^1 + 99 \times 10^0 = 9700 + 980 + 99$   
 $= \dots$

Each JAVA class has hashCode() method.

## How to handle collision?

① Chaining: Allow multiple keys to be stored in a bucket.

\* Allows faster operations.

Implement bucket as

→ array.

→ linked list.

→ search tree.

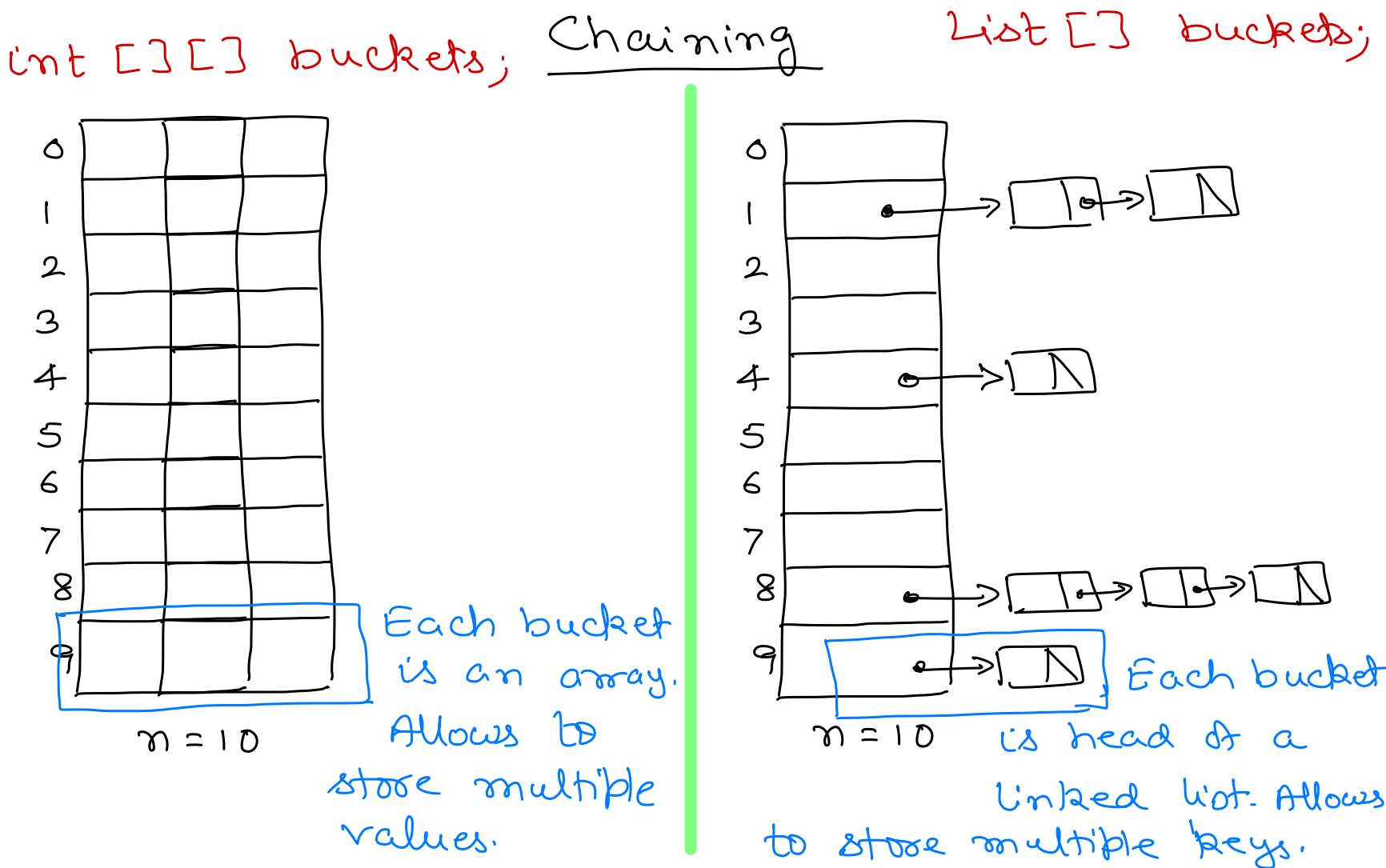
② Open addressing:

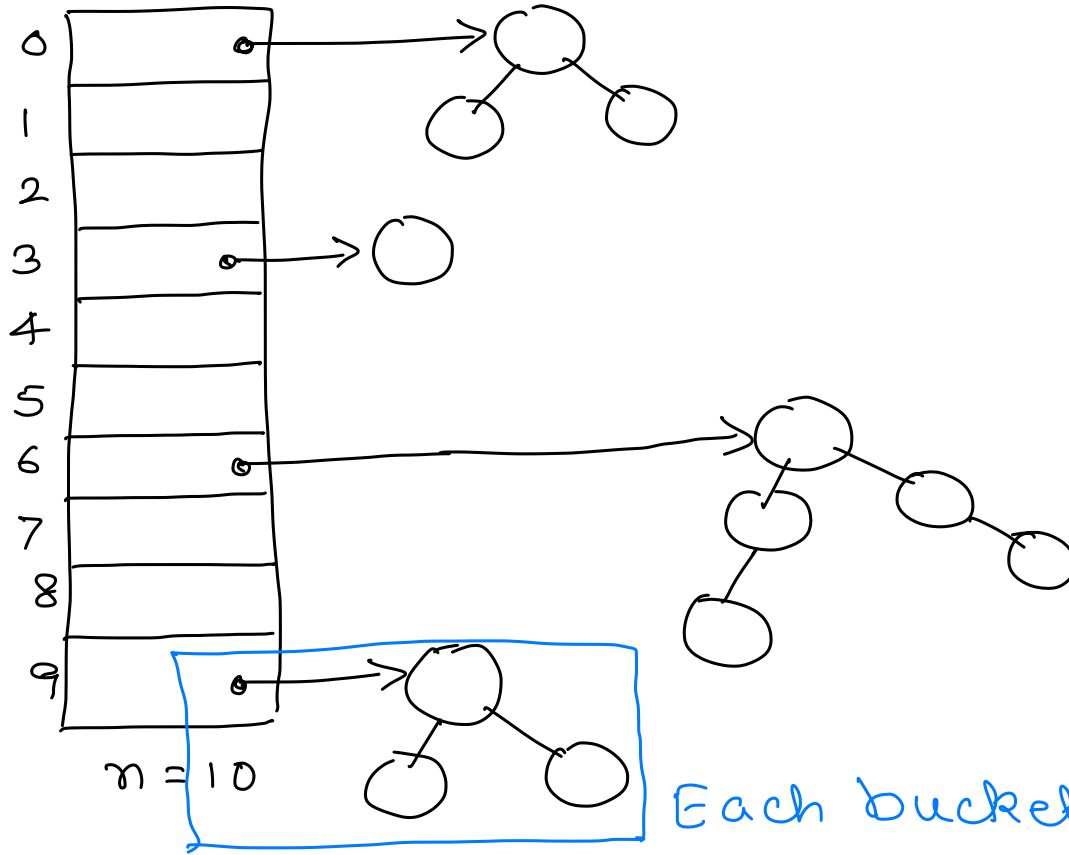
\* uses space efficiently.

→ linear Probing.

→ Quadratic Probing

→ Double Hashing.





BST[] buckets;

Each bucket is head of a BST. Allows to store multiple keys.

Time complexity of insert and search in hash table using chaining.

$$\left. \begin{array}{l} \text{Hash Table Size} = n \\ \text{Bucket Size} = k \end{array} \right\}$$

$n \gg k$   
                      
 $n$  is very very large than  $k$ .

Chaining using BST

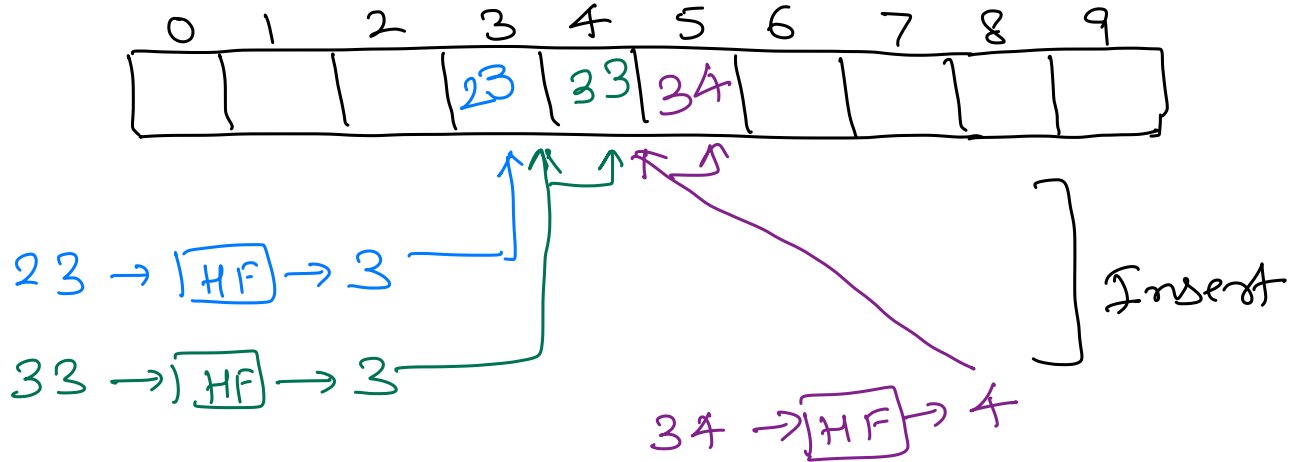
$$\text{Insert} = O(\log_2 k)$$

$$\text{Search} = O(\log_2 k)$$

## Linear Probing

In case of collision, we linearly scan the hash table for next empty bucket and store the new key in it.

It suffers from primary clustering.



We probe for  $n$  times.

If no empty bucket found then

→ Resize the hash table.

→ Rehash the existing keys.

---

Load Factor: How full / empty hash table  
can be before we need to  
resize it.

---

## Quadratic Probing

Uses a quadratic equation to find next empty bucket.

$$\underline{d^2 i + c i + R}$$

$d, c$  are some constant.

$R$  is the key.

$i$  is probe attempt count.

Quadratic probing do not suffer from primary clustering but it suffers from secondary clustering.

## Double Hashing

we use multiple hash functions.

key  $\rightarrow$  HF1  $\rightarrow$  HF2  $\rightarrow$  bucket id